Computer Vision: Laboratory Report

Miguel García Bohigues

May 30, 2024

1 Introduction

This report covers all the details relating to the different tasks proposed in the Computer Vision subject. This document contains five sections with the same structure, each with the name of the assignment. All sections begin by introducing the task at hand and the objective to be achieved. Then the dataset used is described in detail, commenting on its characteristics, peculiarities and some additional information that may be useful, such as the baseline accuracy. Next, the implementation process is detailed, mentioning important information about the network configuration, the optimiser used and the data augmentation techniques. Finally, at the end of each section, the results obtained are commented.

2 Basic implementations

The aim of this first exercise was to familiarise you with PyTorch and some basic computer vision practices such as data augmentation and preprocessing. The task at hand is a classification problem using the CIFAR10 dataset, and was solved by encoding three different network topologies: A VGG-16, a Wide ResNet and a DenseNet. In the following sections we will review some relevant concepts about the dataset, implementation details and the results obtained.

2.1 Dataset: CIFAR10

The CIFAR-10 (Canadian Institute for Advanced Research, 10 classes)[9] dataset is a subset of the Tiny Images dataset[16] and consists of 60000 32x32 colour images. The images are tagged with one of 10 mutually exclusive classes: airplane, car (but not truck), bird, cat, deer, dog, frog, horse, ship, and truck (but not car). There are 6000 images per class, with 5000 training and 1000 test images per class.

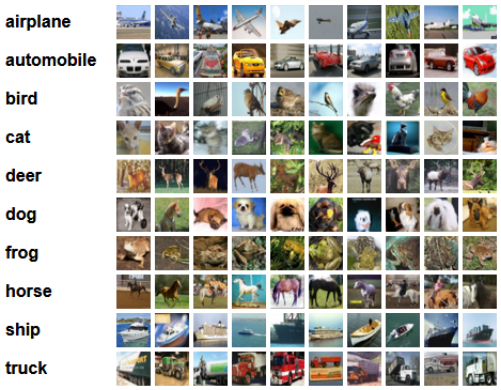


Figure 1: Snippet of 10 examples per class

The dataset website [1] reports a baseline result of $E_{te} = 18\%$ without data augmentation and $E_{te} = 11\%$ with data augmentation. However, with the advent of new network topologies and computer vision techniques, the test error rate has been reduced to only $E_{te} = 0.5\%$ [4].

2.2 Implementation Details

To solve this classification problem, three different network architectures have been developed. First, a VGG [13] model has been implemented to obtain a more accurate baseline according to our application context. Among the different variants presented in the original paper, in this paper we present the D variant with 16 weight layers. Figure 2 shows the resulting topology of the VGG-16.

The same order was followed along the 5 convolution blocks: Convolution + Batch Normalisation[7] + ReLU[11]. The convolution filter is a 3x3 kernel with $stride = 1$ and $padding = 1$. After each convolution block there is a max-pool operation with a kernel of size 2x2 and $stride = 2$, and finally a Dropout[14]

[1] https://www.cs.toronto.edu/ kriz/cifar.html

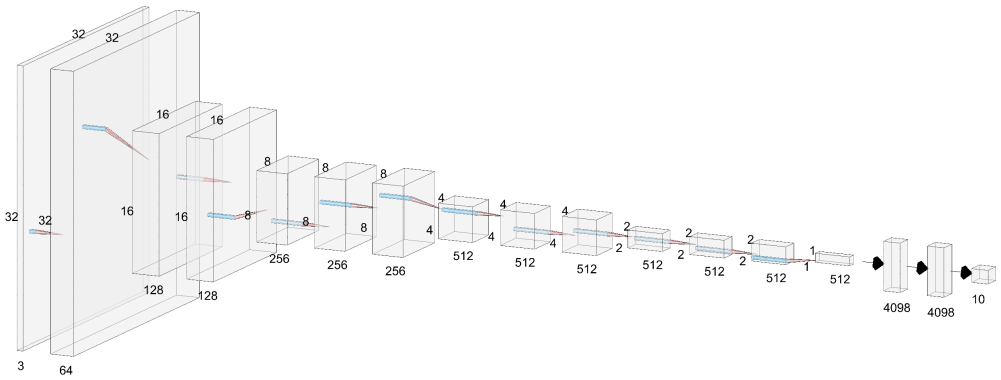


Figure 2: Implemented VGG16 architecture.

operator to avoid over-fitting. The dropout value between convolution blocks is set to $p = 25\%$. Finally, the model presents 3 dense layers, two with 4096 neurons and the last with 10, one for each class. In between there are two drop-out operators with $p = 50\%$.

The second network topology implemented was a fully configurable DenseNet [5] architecture. The basic operator of this network is a BRC "layer" consisting of Batch Normalisation + ReLU + Conv (3x3). At each of the BRCs, the DenseNet concatenates the input with the output via a residual link. As can be seen in Figure 3, the network is composed of four blocks of NxBRC layers, where N reflects the number of times the "layer" BRC is replicated within a block and can be different for each of the blocks. To connect the blocks, there is a transition layer with a (1x1) convolution along a (2x2) average pooling with $stride = 2$ to halve the image size.

Finally, after the last block, we have placed an adaptive average pooling operator that flattens the input into a single vector to be fed to a dense layer that will have 10 neurons as output, one for each class. Note that in the original paper there is a first block consisting of a convolution(3x3) + Batch Norm + ReLU + MaxPool(2x2). It is important to note that we decided to remove the pooling operation due to the small size of the CIFAR10 images. Also, instead of the original four blocks, we decided to reduce them during the experiment to only three, reducing the total number of parameters and thus the overfitting.

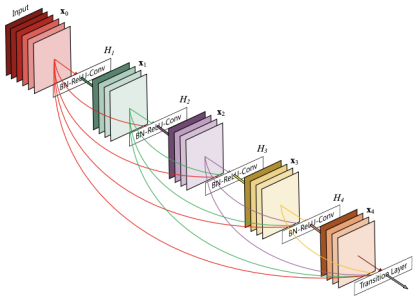


Figure 3: Densenet Structure

The last network topology implemented is the Wide ResNet [17] model. Wide ResNets represent an evolution of the ResNet model, including a widening factor k that provides greater expressiveness and computational efficiency. Like the DenseNet, the Wide ResNet is composed of 4 blocks: The first is a simple 3x3 convolution that projects the input image onto more channels (usually 16). After that, there are 3 groups composed of N blocks of convolution. Figure 4 shows the different types of blocks proposed by the authors in the original paper. Among them, we have chosen the last one, because it has a dropout layer in the middle of the convolution operators, which prevents the network from overfitting. Note that in this case N is the same for all three groups.

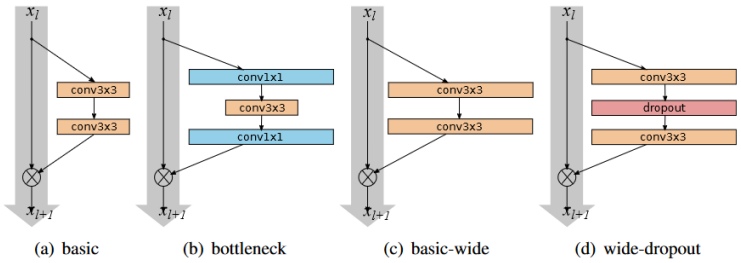


Figure 4: Various Residual blocks used in the original paper. From these, the one implemented in this work is d)

Finally, to complement the network development, we have implemented a configurable data augmentation process with up to 5 transformations:

- Horizontal flip with 50% probability.
- Random rotation between 15 and -15 degrees.
- Horizontal and vertical shift of the image (20% of the width and height).
- Gaussian blur with a 3x3 kernel size.
- An up-sample + central crop transformation.

These transformations are not mutually exclusive and can be combined between them.

2.3 Results

To compare the performance of the three implemented models, we trained all three under the same conditions. We used SGD as optimiser with $lr = 0.1$ and $weight\_decay = 0.0001$. The chosen training criteria was cross-entropy loss, and we implemented a learning rate scheduler based on performance over the validation set. If the system has not improved its score in 10 iterations, the learning rate is reduced by 0.1 ($lr = lr*0.1$). Models have been trained for 150 epochs with an early termination criterion of 35 epochs without improvement on the validation set. Table 1 shows the results obtained with the different networks and data augmentation techniques.

Network	Size	HF	GB	Rot	Shift	Up/Cr	Tr loss	Te loss	Accuracy
VGG	16		x				0.0555	0.4575	85.61
VGG	16						0.0602	0.4552	87.61
VGG	16	x					0.1282	0.3995	88.53
VGG	16	x		x			0.0974	0.3226	90.29
DenseNet	S	x					0.1220	0.3410	90.00
DenseNet	S	x				x	0.0278	0.3144	90.22
DenseNet	S	x		x			0.2760	0.2916	91.41
Wide ResNet	28-10	x					0.0165	0.3547	91.08
Wide ResNet	28-10	x				x	0.0380	0.2898	91.62
Wide ResNet	**28-10**	**x**		**x**			**0.0442**	**0.0253**	**92.23**

Table 1: Results obtained with different networks and data augmentation techniques. S for Densenet denotes three groups each on ewith with 6, 12 and 24 blocks respectively.

Looking at the results we can draw some important conclusions. Firstly, we can see how a simple model without data augmentation suffers greatly from over-fitting to the training samples and is unable to generalise. In addition, we have seen how the inclusion of Gaussian noise has exacerbated this problem, as the network has learned the distribution of this noise and has adapted even more to the training set. On the

other hand, we can see how the inclusion of horizontal flipping and rotation of the images greatly improves the performance, with the best result when both are combined. Finally, if we compare the best performance of each of the models, we can see that indeed wider networks perform better than deeper ones. It can also be seen that the inclusion of residual connections also produces a better result under the same conditions.

3 Gender Recognition

For this exercise, the goal is to develop a model that can classify an image of a person into one of two gender labels: male or female. In section 2 we developed and tested three different network architectures that have proven to work. For this reason, we will use them as the basis for developing the network in this assignment. This assignment seeks two independent goals: On the one hand them model must achieve the maximum possible accuracy without any restrictions, everything counts to achieve a 98% accuracy over the test set. On the other hand a model that achieves at least 95% of accuracy over the test must be developed using less than 100k parameters.

3.1 Dataset: Labeled Faces in the Wild

Labeled Faces in the Wild[6] is a public benchmark for face verification, also known as pair matching. The original dataset contains more than 13k images of faces collected from the web and cropped to a size of 150x150. Each face is labelled with the person's name.

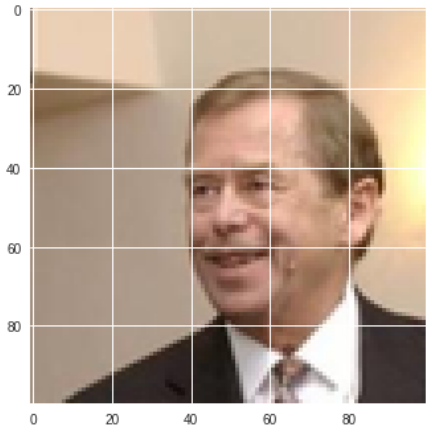


Figure 5: Sample image from our custom LFW [2]

The LFW dataset has been adapted for our problem. Specifically, our images have been cropped to 100x100 instead of the original size. The labels have also been changed. Instead of the name, we have the gender of the person. Figure 5 shows an example of a male face.

In terms of the proportion of the dataset, we have two different sets: one with 10.5k images for training and one with 2.5k for testing. The change in paradigm creates an imbalance in the dataset, as less than a third of the images are female. The fact that one of the classes represents the majority of the samples could drive the model to a local minimum where it always predicts the dominant label.

3.2 Implementation Details

As mentioned in the introduction, for this exercise we have departed from the topologies implemented in Section 2 and made some modifications to them in order to better suit the task at hand.

More specifically, in the case of the VGG-16, the new size of the images produces a map of dimensions 3x3x512 after the last convolution. To connect this map to the fully connected layers, we placed an adaptive average pooling layer that reduces the map to 1x1x512. The dropout values have also been changed, increasing the dropout probability between convolutions to $p = 0.3$.

For the DenseNet case, the architecture has remained almost the same. Unlike the previous implementation, the size of the input images allows us to introduce the first max pooling operator to reduce the dimension of the image. And also allows us to use the four blocks of convolutions as in the original paper, so the configuration for the 'S' size is $[6, 12, 24, 16]$.

[2]Image extracted from the professor's GitHub

Finally, the structure of the Wide ResNet remained the same, with only a few changes to the stride and padding of some convolutions. More specifically, the stride of the first convolution block was set to $stride = 2$ with $padding = 0$ to reduce the dimension of the image. For the second convolution, the stride was increased to $stride = 2$, again to reduce the dimension. Finally, instead of a constant average pooling layer, we placed an adaptive average pooling layer that reduces the map to a single vector to feed the linear layer.

For the case with a limited number of weights, we followed the idea presented in [1] and developed a fully sequential convolution neural network composed of three blocks. In the first, there are two (3x3) convolutions that expand the image to 32 dimensions. The second block then applies two more 3x3 convolutions that preserve the depth. Finally, the last convolution block applies a final pair of 3x3 convolutions, increasing the depth of the image to 64 dimensions. After each of these blocks there is a max pooling operator that halves the dimension of the image.

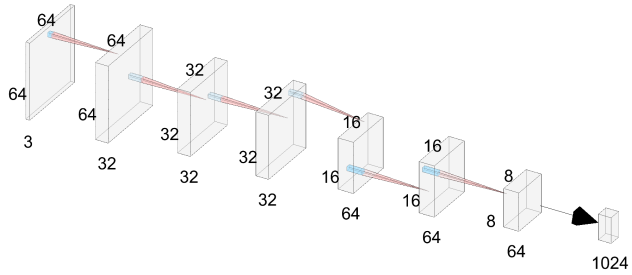


Figure 6: Architecture of the weight restricted net. Image sizes displayed for an input image of 64x64.

We tried two different approaches to the constrained problem. The first was to use 64x64 images, which became 8x8x64 after the last pooling operator. To reduce this dimension, we applied an additional average pooling operator which reduced the size to 4x4x64 and finally flattened the result. The second experiment was with 32x32 images. As running these images through the network produced a map of size 4x4x64, there was no need for the additional average pooling in this case. We simply flattened the map and applied it to the linear layer.

Regarding the data augmentation techniques, using our experience from the previous exercise we adapted them:

- We removed the gaussian blur since it proved to worsen performance.
- We reduced the range of the shifting to only the 10% of the image width and height. To avoid cutting the face or hair.
- We got rid off the up-scaling and maintained only a center crop with customised size.

The previous transformation that have not been cited (horizontal flip and rotation) have remained the same.

3.3 Results

In order to draw conclusions and better understand the effect of different hyperparameters and transformations, all models were trained for a maximum of 300 epochs, with an early stopping criterion of improvement on the test set over 50 epochs. Also, the learning rate will decay by $lr = lr * 0.1$ if the model has not improved after 15 epochs. With this training procedure we have tried two different optimizers, SGD and Adam[8]. The later gave significantly better results, so the main investigation was carried out with it.

Data augmentation techniques were difficult to apply in this case. Their absence led to a huge overfitting of the models, but too much variation in the data led to a deadlock in the local minima, which always predicted the dominant class (male). To find the better combination we tried no transformation, horizontal flip, rotation and shift. All separately and then together in pairs and all three. Of all the combinations, the best results were obtained using only the horizontal flip. This seemed to give enough variability without making the task too difficult.

In addition, we introduce three different cases to cope with the imbalance of the dataset. The first case corresponds to leaving the dataset as it is, imbalanced. The second case makes use of a weighted loss to

penalize more when the model predicts "Male" and "Female" is the correct label. The third and final case uses balance at batch level. By doing this we ensure that the model always processes the same amount of images for the two labels within a batch. Those cases are labelled as "imb", "loss" and "batch" respectively.

Table 2 shows a comparison that highlights the process used to achieve the desired performance. By looking at the first three rows of each model, we can compare the best data weighting technique. As it can be seen, among the three of them, the worse is always the batch weighting. One possible cause for this to happen could be the amount of times that the *Female* images are revisited. As too much data augmentation leads the network to fail into the local minimum, the network may be seeing the same images over and over again, failing to generalise. For the remaining two data balancing techniques, we see that treating the data set as it comes seems to be advantageous. As the loss functions are different due to the weighting process, we cannot focus on that metric, but by looking at the final accuracy of all three models, we see how letting the dataset imbalanced provides better results.

	Network	**Size**	**HF**	**DB**	**Wd**	**Lr**	**Tr loss**	**Te loss**	**Accuracy**
	VGG	16		loss	1e-4	1e-3	0.0023	0.1086	96.45
	VGG	16		batch	1e-4	1e-3	0.6226	0.6852	77.49
	VGG	16		imb	1e-4	1e-3	0.0016	0.0887	96.64
	VGG	16	x	imb	1e-4	1e-3	0.0011	0.0615	97.66
	VGG	16	x	imb	1e-3	1e-3	0.0085	0.0548	97.81
(*)	**VGG**	**16**	**x**	**imb**	**1e-3**	**5e-4**	**0.0028**	**0.0532**	**98.03**
	DenseNet	S		loss	1e-4	1e-3	0.0041	0.1623	94.74
	DenseNet	S		batch	1e-4	1e-3	0.0016	0.3224	93.13
	DenseNet	S		imb	1e-4	1e-3	0.0009	0.1177	97.13
	DenseNet	S	x	imb	1e-4	1e-3	0.0017	0.1113	96.34
	DenseNet	S	x	imb	1e-3	1e-3	0.0011	0.0845	97.09
	DenseNet	S	x	imb	1e-3	1e-5	0.0047	0.679	97.47
	Wide ResNet	28-10		loss	1e-4	1e-3	0.0016	0.1653	93.35
	Wide ResNet	28-10		batch	1e-4	1e-3	0.0006	0.3475	92.11
	Wide ResNet	28-10		imb	1e-4	1e-3	0.0006	0.1169	95.81
	Wide ResNet	28-10	x	imb	1e-4	1e-3	0.0009	0.0953	96.90
	Wide ResNet	28-10	x	imb	1e-3	1e-3	0.0006	0.0870	97.02
	Wide ResNet	28-10	x	imb	1e-3	1e-5	0.0032	0.0960	96.87

Table 2: Results over the LFW dataset with different hyperparameters. Wd stands for weight decay and Lr for learning rate. (*) indicates that the model complies with the condition of at least a 98% of accuracy on the rest set.

In addition, as mentioned above, we can see how the inclusion of a horizontal flip transformation improves the performance of the VGG and Wide ResNet models. For the DenseNet, the direct comparison results in a decrease in performance, but comparing further modification of the hyperparameters, with and without horizontal flip, resulted in consistently better results when horizontal flip was used.

Finally, since the better performance of each of the models was very close to the target, we decided to modify the weight decay regularisation parameter, which resulted in better results. Finally, we lowered the initial learning rate until we reached the optimal performance highlighted in bold in the table.

Changing now the topic of study and focusing on the network with a limited number of parameters, we followed a similar training methodology. Starting from the best hyperparameter combination obtained before (Adam as optimiser, $lr = 1e-4$ and $weight\_decay = 1e-3$), we ran the training for 150 epochs with an early stopping criterion of 35 epochs if no improvement is seen over the test set.

As we can see in the table 6, we first made a comparison between the different data balancing methods. Contrary to what we saw under the previous conditions, here we see that all three methods perform similarly for the 64x64 images. Due to the simplicity of the network, adding data enhancement techniques does not significantly improve the accuracy. It only does the training more complex, especially when adding shift transformations. For this reason, we limited the runs to one transformation at a time.

For the smaller images we see a different trend. Here, the inclusion of a weighted batch improves compared

	Parameters	Im. size	HF	Rot	Shift	DB	Accuracy
(*)	92770	64x64				imb	96.15
(*)	92770	64x64				loss	96.07
(*)	92770	64x64				batch	96.15
(*)	**92770**	**64x64**	**x**			**imb**	**96.90**
(*)	92770	64x64		x		imb	96.19
(*)	92770	64x64			x	imb	95.15
	86626	32x32				imb	93.88
	86626	32x32				loss	92.41
	86626	32x32				batch	94.52
	86626	32x32	x			batch	93.43
	86626	32x32		x		batch	85.61
	86626	32x32			x	batch	84.18
(*)	86626	32x32	x			imb	95.47
	86626	32x32		x		imb	92.26
	86626	32x32			x	imb	92.05

Table 3: Parameter-limited network results over the test set. DB stands for data balance method, which can be Imbalanced, weighted loss or weighted batch. (*) indicates that the model complies with the condition of at least a 95% of accuracy on the rest set.

to the other two weighting methods when no data augmentation is applied. However, once we have added the data enhancement techniques, its performance has decreased with respect to the unbalanced case.

4 Bi-linear Models

This section corresponds to the third assignment. The aim here is to develop a bi-linear model [10] for fine-grained classification, which typically involve discriminating between categories that share a common structure but differ in subtle ways. As a result, the final model is expected to achieve at least 65% accuracy on the test set.

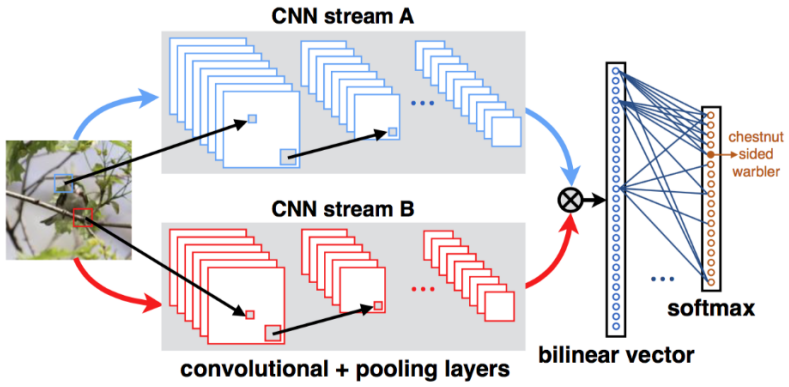


Figure 7: Example of feature combination using bi-linear CNN

Bi-linear models work by performing an outer product of the maps along the depth dimension. For this to be possible, both feature sizes must be equal. By performing this operation, bi-linear allows the outputs of each of the feature extractors to be conditioned on each other by considering all of their pairwise interactions.

4.1 Dataset: Car Models

For this task, the available dataset consists of car images with a size of 250x250. The number of samples is limited because we only have 1575 images available, 785 of which are for testing. This leaves only 791 images for training the network. The dataset contains 20 different cars that the model needs to distinguish. Below are a few different images sampled at random.

(a) Hummer (b) AUDI (c) Aston Martin

Figure 8: Three different cars and their corresponding class

4.2 Implementation Details

Due to the lack of training samples, it becomes very difficult to train from scratch a full CNN. To overcome this problem, we decide to make use of the pre-trained models available at the PyTorch library. We have used them to build out bi-linear model by adopting the "fully shared" approach presented in the original

paper. This approach promotes the use of the same network for the two feature extraction. To introduce variance in the outer product, we have placed a Dropout layer with $p = 30\%$ for each of the features.

Among all the models available at the PyTorch repository, we have selected the following three for having a deep understanding of their architecture. We think that their similarity in performance, a priori, on the ImageNet dataset makes them comparable in this specific task.

- VGG-16: a VGG with 16 layers and batch norm regularization. This model has a 73.36% top-1 accuracy on ImageNet.
- ResNet-50: The medium of the ResNet family. This model has a 76.13% top-1 accuracy on ImageNet.
- DenseNet-121: the smaller of the DenseNet models. It has a 74.34% top-1 accuracy on ImageNet.

The procedure for building the bi-linear is the same for all pre-trained models. The first step is to load the full model and initialise the weights with those from the ImageNet pre-training. Then, we remove the classification layer and select one of the previous convolution layers as the feature extractor. More precisely, from the VGG architecture, we have tried with the last ReLU (ReLU-43), the previous one (ReLU-40) and some others more internal (ReLU-37 .. ReLU-30). For the ResNet-50 and the DenseNet, we have experimented with removing only the classifying layer (-1) and also removing the last convolution block (-2).

With the representation obtained from the aforementioned layers, we can perform the outer product, which results in a map of size BxDxD, where B is the number of images in the batch and D is the number of features in the depth dimension. Then, we apply a normalisation procedure that divides the result by the signed square root and applies L2 regularisation[3]. Finally, we have to flatten the resulting map into a vector and feed it into a linear classification layer.

4.3 Results

The training procedure for this exercise was done with an Adam optimiser, $lr = 0.1$ and $weight\_decay = 0.001$. After N steps (where N is an adjustable parameter), the learning rate was reduced to $lr = 0.0001$. No data augmentation technique was used in this case

	Network	Batch	F.E. Layer	Epochs	Unf. epoch	Initial lr	Unf. lr	Accuracy
(*)	**vgg**	**128**	**43**	**200**	**150**	**0.1**	**0.0001**	**79.85**
(*)	vgg	128	43	200	100	0.1	0.0001	79.46
(*)	vgg	64	43	100	75	0.1	0.0001	78.95
(*)	vgg	64	40	100	75	0.1	0.0001	78.57
(*)	vgg	64	37	100	75	0.1	0.0001	75.64
(*)	vgg	64	33	100	75	0.1	0.0001	70.92
	vgg	64	30	100	75	0.1	0.0001	64.16
(*)	resnet	64	-1	200	100	0.1	0.0001	73.85
(*)	resnet	64	-2	200	100	0.1	0.0001	71.94
(*)	resnet	64	-3	200	100	0.1	0.0001	68.11
(*)	densenet	64	-1	300	150	0.1	0.0001	77.81
(*)	densenet	64	-1	200	150	0.1	0.0001	76.66
(*)	densenet	64	-1	200	100	0.1	0.0001	76.15

Table 4: Comparison of layer selection as feature extractor. (*) indicates that the model complies with the condition of at least a 65% of accuracy on the rest set.

As the table 4 shows, for the VGG there is no major difference in the choice of feature extractor between the last ReLU and its immediate predecessor. However, as we move up in the network topology, we notice a drop in performance as we approach to the source of information. Something similar can be seen with the ResNet, as its performance also decreases as we select layers closer to the input.

Regarding the unfreeze time, we observed that between 100 and 150 epochs it might be the best time to unfreeze the network and adapt the pre-trained weights to the task at hand. Earlier unfreezing also gives

good results, but systems tend to improve by about 2 points of accuracy if the unfreezing is done when the linear layer is more competent at the task. Nevertheless, we observed that delaying the unfreezing process beyond the 150 epochs do not produce better results.

5 Image Colorization

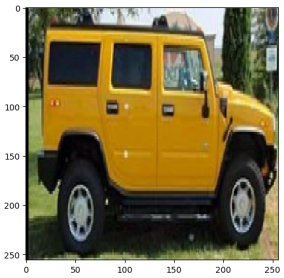
Colouring gray images can have a big impact in a variety of areas, such as remastering historical images or reducing the amount of information shared in video calls. Colourisation is usually done manually in Photoshop and it can take up to a month to colourise a single image due to the intensive required research.

For this reason, it has become a focus of research in computer vision. It requires a new paradigm because the model takes an image as input and the output must also be an image. The model no longer classifies what is inside, but generates the colours it can have. It is also an interesting task because there is more than one correct answer, as the model could paint a car with many different colours and all of them could be correct.

Our goal in this task is to develop an image-to-image model that consists of an encoder-decoder architecture and uses an Inception ResNet [15] to improve its results. As a result of the training phase, the model should be able to colourise at least parts of images.

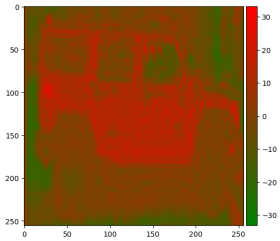
5.1 Dataset

Following the professor's guidelines, we consulted the reference GitHub repository and selected their training images. These are 10 images of different people in different contexts. After some experimentation, we concluded that the training set was not sufficient for our task. Therefore, we selected another dataset that could provide more samples. Since we wanted the images to be similar, we chose the car dataset used for training the bi-linear model in the previous section. From this we only used the training images.

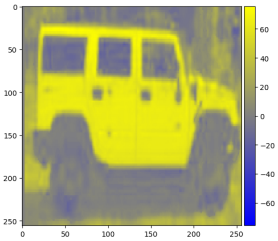


(a) Original image

(b) Luminance layer



(c) A channel layer



(d) B channel layer

Figure 9: First sample of the dataset

There are different representations of a coloured image: RGB, MCYK, LAB, etc. Of these, the LAB encoding may be the most suitable for the task of colourisation, as it is made up of three layers. Firstly, luminance, then A for red and green, and finally B for blue and yellow. This codification is practical because the model can start from the luminance layer and predict the other two, rather than predicting the three RGB layers. Figure 9 shows the representation of the whole image and each one of the different channels.

5.2 Implementation details

As stated in the introduction of this section, the goal is to develop an encoder-decoder architecture enhanced with an Inception ResNet. The basic idea is to colourise the image by performing four steps. The first

two are performed independently and then combined to feed the third step. The first step corresponds to obtaining a vector $v \in \mathbb{R}^{1000}$ representation of the image that contains the semantic information within. This was done by applying a forward pass over an Inception-V3 model pre-trained with ImageNet. The second step is to compress the image by applying a series of convolutional layers. In our case, we have followed the architecture proposed in [2], which can be seen in Figure 10, with the only difference that our images have been scaled to 256x256, so that the internal map after the last encoder convolution will have a size of 32x32x256.

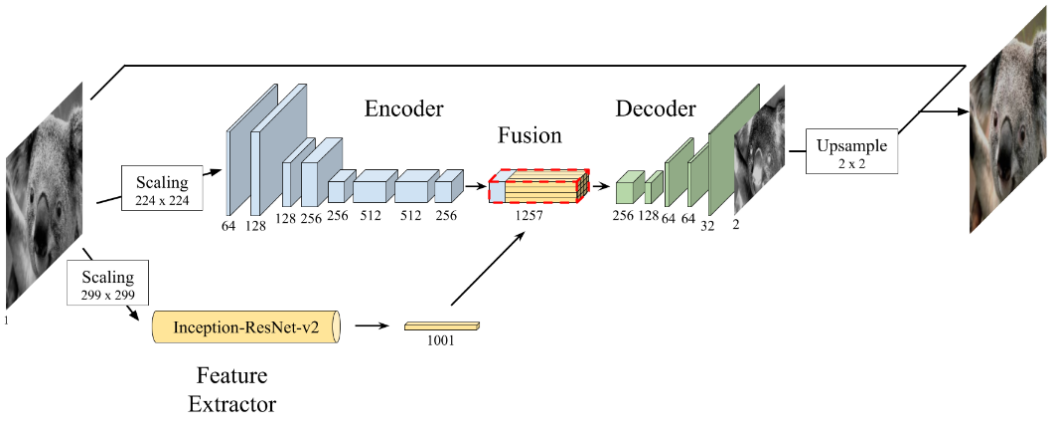


Figure 10: An overview of the model architecture presented at [2]

The next step corresponds to the fusion of the Inception ResNet output and the encoder output. The fusion is first performed by repeating v for each of the pixels that make up the internal map obtained by the encoder. The result of this repetition is a map of size 32x32x1000. It is then concatenated with the internal map in the channel dimension, resulting in a 32x32x1256 map.

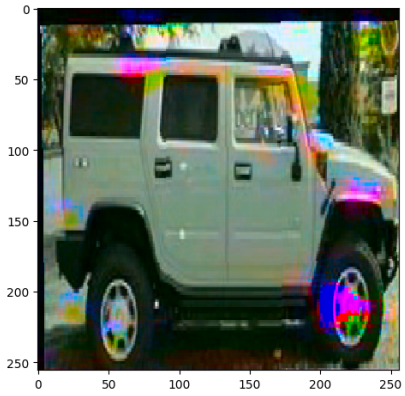
Finally, the last step is to gradually upsample the map while reducing the number of channels. The output layer should have only 2 channels, one for the A and the other for the B.

The system was trained for 150 epochs by minimising the mean squared error (MSE) between the generated maps and the ground truth. The chosen optimiser was Adam with $lr = 1e - 5$ and a weight decay of $1e - 4$. As with previous models, we set a learning rate scheduler that reduces the current learning rate by one tenth if the model has not been able to improve its performance after 25 epochs.

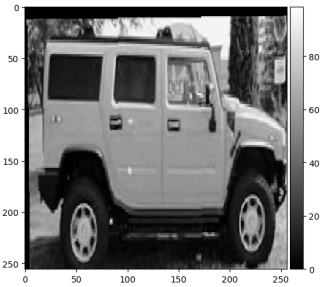
5.3 Results

During the training process, the model successfully converged to a loss of less than 0.1 in only four epochs. Thereafter, the value of the loss decreased almost monotonically until it reached the minimum value of 0.0058 after the full 150 epochs. To visualise the performance of the model, we have selected the first ten images of the training set and displayed the predicted maps.

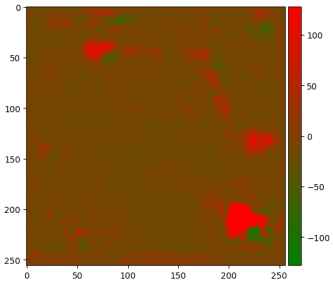
Figure 11 shows the result of running figure 10 through the model. As you can see, the model has successfully coloured the tree and some of the surroundings of the car. The car itself was painted with a light grey colour. This may be caused by the definition of the loss function, as the model may be trying to reduce the MSE by predicting light colours that do not have strong properties. The image also shows how the model includes some strong pink and cyan areas. This is an issue that we have not been able to address. If we look at the predicted channel layers, we can see how the top shape of the truck and the wheels are partially visible.



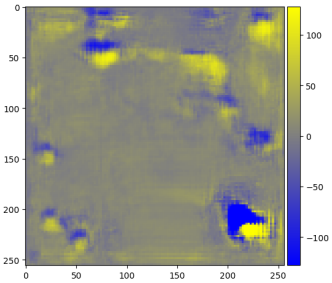
(a) predicted image colours



(b) Luminance layer



(c) Predicted A channel layer



(d) Predicted B channel layer

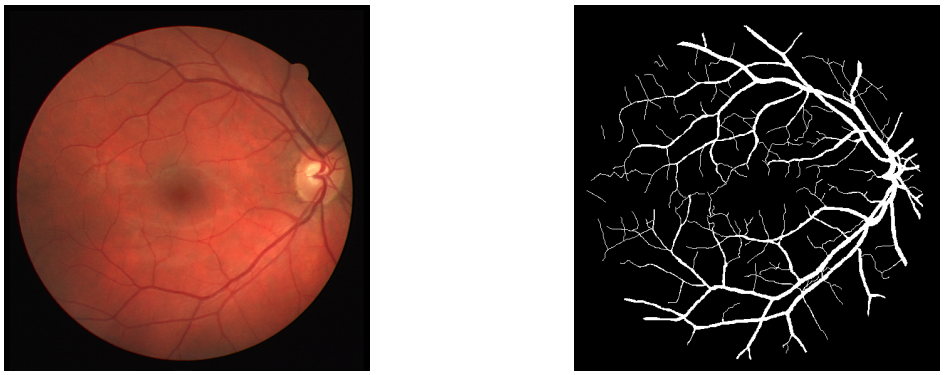
Figure 11: Generated colors for the first sample of the dataset

6 Image Segmentation

Image segmentation is a fundamental process in digital image processing where an image is divided into multiple segments or regions based on pixel characteristics. This technique is widely used in various fields such as medical imaging, autonomous driving, etc. The final task of this topic corresponds to the implementation of a U-net [12]. The final model must be able to segment eye images, highlighting the veins inside the retina.

6.1 Dataset

The data set for this task corresponds to 20 different images of a retina. Each one has its corresponding mask that outlines the veins. Figure 12 shows the first example of the dataset together with its mask.



(a) Original image of the retina

(b) Mask of the retina

Figure 12: First sample of the dataset[3].

For this problem, 20 images is a very limited dataset. Therefore, our goal is to achieve a correct segmentation of the training data set. In this case, we are not aiming for a model that can generalise. Therefore, we will use the 20 images for both training and testing.

To add some randomness to the samples, we have added a data augmentation process that allows images to be shifted horizontally and vertically. The total shift distance in both directions is set to 1% of the total width and height of the image. A rotation of 1 degree has also been added. The sizes of the transformations are very small in order not to cut the image too much and to avoid possible losses.

6.2 Implementation details

The name of the U-net comes from its shape. As figure 13 shows, this network has a U-shaped form due to its compression and decompression process. More precisely, the compression process is performed by applying four encoder blocks. Each of these blocks consists of a sequence of convolution(3x3) + batch norm + ReLU + convolution(3x3) + batch norm + ReLU. In this case, the convolutions do not reduce the dimensionality of the image. Instead, they only increase or decrease the number of channels. The dimensionality reduction is performed in the encoder block by applying a MaxPool(2x2) layer.

The decompression process follows the same principle, but with an increase in the size of the image. It consists of four decoding blocks that apply a transposed convolution with a kernel of size 2 and stride 2 to upscale the image. The output of the encoder at this level is then concatenated with the result of the previous transposed convolution, and the resulting map is then passed through the same Convolution(3x3) + Batch Norm + ReLU + Convolution(3x3) + Batch Norm + ReLU sequence as in the encoder.

In the specific case of this task, the input images are cropped to a size of 512x512. They are reduced to 32x32 in the last encoder block and then restored to their original size in the last decoder block. Note that in our case, unlike the image, the size of the maps does not change between convolutions. The size only changes when a max pooling operator or a transposed convolution is applied. Finally, since our task only

[3]Image extracted from the professor's GitHub

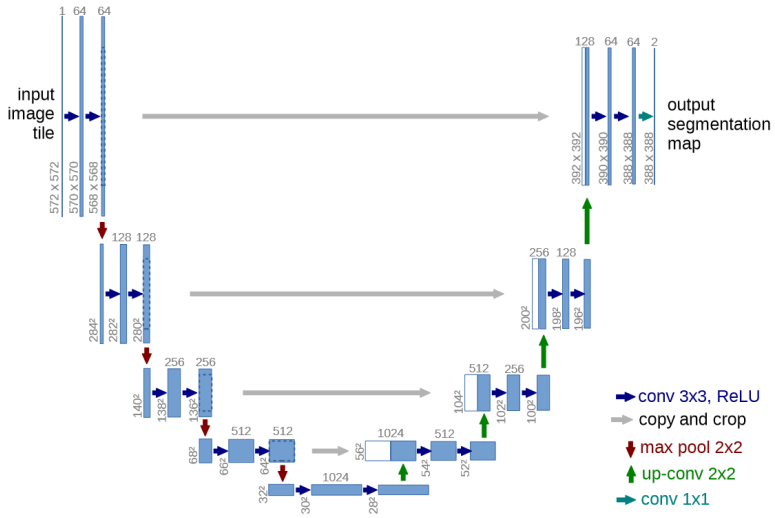


Figure 13: U-Net architecture[12]

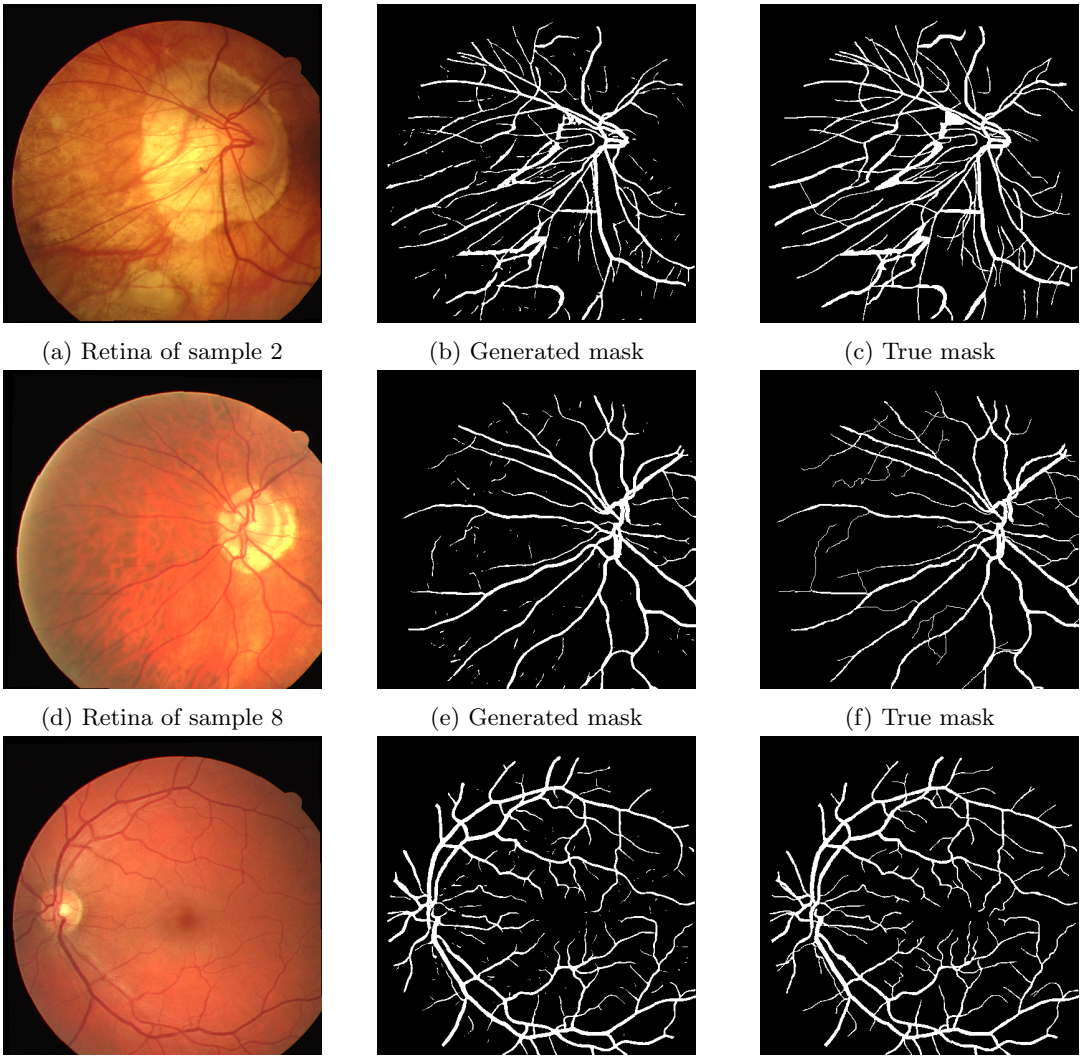
requires the segmentation of a single entity, the veins, we have placed a single channel at the output, which will be used as a pixel-wise binary classifier.

6.3 Results

For the training phase we used a special configuration for this task. Due to the computational limitations of the environment, the model can only process a batch of 2 images at a time. Following the guidelines given in the original paper, we used SGD as the optimiser with a high momentum (0.99), so that the previously seen samples determine the update in the current optimisation step. The loss function was set to binary cross entropy.

The model was trained for 500 epochs with an initial $lr = 1e - 3$. As in previous cases, we have set a learning rate scheduler that reduces the learning rate by a tenth ($lr = lr * 0.1$) if the model has not improved its performance after 25 epochs. Although we set an early termination condition after 75 epochs without improvement, the model did not meet this condition and training was terminated after 500 epochs with a final training loss of 0.0338.

Figure 14 shows three examples of the segmentation of three different retinas. As can be seen, the predicted mask is very accurate compared to the real one. However, it can also be seen that there are some noisy regions on the predicted mask corresponding to very small veins.



(a) Retina of sample 2 (b) Generated mask (c) True mask

(d) Retina of sample 8 (e) Generated mask (f) True mask

(g) Retina of sample 14 (h) Generated mask (i) True mask

Figure 14: Three different cars and their corresponding class

7 Conclusions

This report explains how we approached each of the proposed tasks. For each of them, there were some underlying problems that we faced and learned from. Firstly, we understood the principles behind three of the most widely used computer vision network topologies, such as VGG, DenseNet and Wide ResNet. Second, we faced a classification problem where we had to achieve peak performance while dealing with an unbalanced dataset and model size constraints. Thirdly, we learned how to mix the representation of two networks using the bi-linear scheme, and we applied this to a fine-grained classification problem with excellent results. Next, we tackled a different problem: image colouring, where we encountered some generalisation problems and additional difficulties. Despite the limited performance, we learned a lot about what's behind Torch's transformations and channel management. Finally, we faced a final classification problem in an image-to-image paradigm, where we learned how to implement a U-net for biological image segmentation.

References

[1] Grigory Antipov, Sid-Ahmed Berrani, and Jean-Luc Dugelay. "Minimalistic CNN-based ensemble model for gender prediction from face images". In: *Pattern Recognit. Lett.* 70 (2016), pp. 59–65. URL: https://api.semanticscholar.org/CorpusID:13034475.

[2] Federico Baldassarre, Diego González Morín, and Lucas Rodés-Guirao. *Deep Koalarization: Image Colorization using CNNs and Inception-ResNet-v2*. 2017. arXiv: 1712.03400 [cs.CV].

[3] Corinna Cortes, Mehryar Mohri, and Afshin Rostamizadeh. *L2 Regularization for Learning Kernels*. 2012. arXiv: 1205.2653 [cs.LG].

[4] Alexey Dosovitskiy et al. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. 2021. arXiv: 2010.11929 [cs.CV].

[5] Gao Huang et al. "Densely connected convolutional networks". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2017.

[6] Gary B. Huang et al. *Labeled Faces in the Wild: A Database for Studying Face Recognition in Unconstrained Environments*. Tech. rep. 07-49. University of Massachusetts, Amherst, 2007.

[7] Sergey Ioffe and Christian Szegedy. "Batch normalization: Accelerating deep network training by reducing internal covariate shift". In: *International conference on machine learning*. pmlr. 2015, pp. 448–456.

[8] Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. 2017. arXiv: 1412.6980 [cs.LG].

[9] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. "Imagenet classification with deep convolutional neural networks". In: *Advances in neural information processing systems* 25 (2012).

[10] Tsung-Yu Lin, Aruni RoyChowdhury, and Subhransu Maji. "Bilinear CNN Models for Fine-Grained Visual Recognition". In: *2015 IEEE International Conference on Computer Vision (ICCV)* (2015), pp. 1449–1457. URL: https://api.semanticscholar.org/CorpusID:1331231.

[11] Vinod Nair and Geoffrey E Hinton. "Rectified linear units improve restricted boltzmann machines". In: *Proceedings of the 27th international conference on machine learning (ICML-10)*. 2010, pp. 807–814.

[12] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. *U-Net: Convolutional Networks for Biomedical Image Segmentation*. 2015. arXiv: 1505.04597 [cs.CV].

[13] Karen Simonyan and Andrew Zisserman. *Very Deep Convolutional Networks for Large-Scale Image Recognition*. 2015. arXiv: 1409.1556 [cs.CV].

[14] Nitish Srivastava et al. "Dropout: a simple way to prevent neural networks from overfitting". In: *The journal of machine learning research* 15.1 (2014), pp. 1929–1958.

[15] Christian Szegedy et al. *Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning*. 2016. arXiv: 1602.07261 [cs.CV].

[16] Antonio Torralba, Rob Fergus, and William T Freeman. "Tiny images". In: (2007).

[17] Sergey Zagoruyko and Nikos Komodakis. *Wide Residual Networks*. 2017. arXiv: 1605.07146 [cs.CV].